From two example codes to measure inter-NUMA-node bandwidth and latency to OpenMP memory allocators

Lecture notes — HPC1 - Università di Trieste, A.Y. 2025-2026
Luca Tornatore, luca.tornatore@inaf.it

Lecture notes, 22 April 2026. These notes cover the second part of the lecture; the first part — the OpenMP Memory Model — is the subject of a dedicated lecture note.
*This note is about controlling **where** the memory is, in terms of NUMA placements and more general memory proprties, using low-level techniques and high-level OpenMP tools.*

From two example codes to measure inter-NUMA-node bandwidth and latency to OpenMP memory allocators
1. The target platform
2. The strategy
3. Adding a direct probe of memory pages allocation
4. Direct allocation with `numalib`
5. Binding pages
6. Requiring large pages
7. Measuring the latency
8. What's next

Appendix: the `stream` routine details

1. The target platform

We target the same 4-sockets Skylake-SP (Gold 5118) node that we used to test the π-greek performance in the note "OpenMP Performance — A Case Study with Monte Carlo π" (Sec. 5.2).
I report here the diagram that illustrates the architecture:

```
┌──────────────────┐      UPI      ┌──────────────────┐
│    Socket 0      │◄──────────►   │    Socket 1      │
│  12 cores, no HT │               │  12 cores, no HT │
│  16.5 MB L3      │               │  16.5 MB L3      │
│  (non-inclusive) │               │  (non-inclusive) │
│  6 × DDR4-2400 ch │              │  6 × DDR4-2400 ch │
│  Internal: mesh  │               │  Internal: mesh  │
└────────┬─────────┘               └────────┬─────────┘
         │ UPI                              │ UPI
         │                                  │
┌────────┴─────────┐               ┌────────┴─────────┐
│    Socket 3      │◄──────────►   │    Socket 2      │
│  12 cores, no HT │     UPI       │  12 cores, no HT │
│  16.5 MB L3      │               │  16.5 MB L3      │
│  (non-inclusive) │               │  (non-inclusive) │
│  6 × DDR4-2400 ch │              │  6 × DDR4-2400 ch │
│  Internal: mesh  │               │  Internal: mesh  │
```

```
   └──────────────────┘            └──────────────────┘


         Total: 48 cores / 48 threads (HT off), 4 sockets
         Inter-socket: ring of 2 UPI links per socket
         Socket 0 ↔ Socket 2: 2 hops (via Socket 1 or Socket 3)
```

The Gold 5118 has only 2 UPI links, so in a 4-socket configuration the sockets form a *ring* — not fully connected. To get from socket 0 to socket 2 you must pass through socket 1 or 3: **2 hops**. That roughly doubles the worst-case coherence latency.
The memory distance matrix as reported by `numactl -H` is:

```
     S0   S1   S2   S3
___ ________________
S0   10   21   21   21
S1   21   10   21   21
S2   21   21   10   21
S3   21   21   21   10
```

This matrix reports informations about the *latency* for accessing memory, not about the bandwidth. Those informations are provided in the System Locality Information Table (SLIT) tables, they are not a measure. So, in this lecture we want to solve the problem of *measuring* both the bandwidth and the latency of the memory access.

We will progress incrementally, adding techniques and details as reported in the following table:

tag	Version	Content
v0	`test_bandwidth.base.c`	the baseline code
v1	`test_bandwidth.probe_allocation.c`	v0 + inspection of where the pages are physically placed
v2	`test_bandwidth.numa_alloc.c`	v1 + explicity numa allocation beyond the first-touch policy assumption
v3	`test_bandwidth.pages_bindng.c`	v2 + page binding request (to avoid pages migration)
v4	`test_bandwidth.large_pages.c`	v3 + kind request to have large pages (`> 4KB`) and/or to merge the contiguous small pages

Note that while implementing new features, you can activate them at command line by using the command-line options `-p, -n, -b, -l`.

2. The strategy

Our basic strategy is implemented in the accompanying file `test_bandwidth.base.c`. What we do is the following:

1. set `OMP_PLACES=sockets`, and test in from the code, rejecting anything different;
2. get how many `places` there are, and open a parallel region with `num_threads(nplaces)`

`proc_bind(spread)` clauses, so to have as many threads as sockets and 1 thread per socket;

3. allocate the same memory bunch, larger than L3, per thread, assuming first-touch policity;
4. running the following experiment (pseudo-code):

```
for ( int socket = 0; socket < nsockets; socket++ )
    for ( int target = 0; target < nsockets; target++ )
        measure_bandwidth_btw_sockets( socket, target );
```

and reporting the obtained *measured* bandwidth matrix.

The implementation is pretty straightforward.
The actual bandwidth measure is conducted by a `stream`-like routine: for the implementation details, refer to the Appendix.

On our platforms this gives back the following measured matrix:

```
        S0      S1      S2      S3
-------------------------------------
S0    | 11      4.9     5       5.1
S1    | 5       10      4.8     4.9
S2    | 4.8     4.8     11      4.9
S3    | 4.8     4.7     5       12
```

→ actually, this result has been obtained with `numa_balancing` set off; see Sec. 5.

3. Adding a direct probe of memory pages allocation

If we wanted to actually test where our memory pages live, we can read the `/proc/self/numa_maps` file exposed by the kernel, looking for allocations larger than the per-thread allocation.
Details are given in Sec. 6 of the `lecture_notes–numa_memory_control` file, which gives a primer about how to use `numalib` library on Linux systems.
The implementation is in the routine `probe_memory_alloc( )`

4. Direct allocation with `numalib`

Up to this point we are assuming that the system's policy for memory pages allocation is first-touch. However, we may want to be independent on that, and use an explicit allocation. We can achieve that by using the function call
`numa_alloc_onnode()`
from `numalib`.
With `numa_alloc_onnode`, the physical pages are bound to the specified node by `mbind` *before* any touch. It doesn't matter which thread initialises them — they stay on the right node.
This is immune to NUMA balancing, to allocator reuse, and to any other kernel heuristic.
The call is implemented in v2 `test_bandwidth.numa_alloc.c`.
Remind: compile with `-lnuma` to link the `numalib`.

5. Binding pages

In Sec. 2 I have cheated a bit. The memory matrix obtained at the very first test was

```
        S0      S1      S2      S3
---------------------------------------
S0    | 12      6       7.4     6.4
S1    | 5.9     5.7     4.9     4.9
S2    | 4.9     4.8     5.3     4.8
S3    | 4.8     4.8     4.8     7.4
```

Pretty much different, awfully asymmetric and with a non-homogeneous diagonal values.
That was due to the fact that the kernel's `numa_balancing` parameter was set to 1, which means "migrate pages when you spot that a thread is consistently accessing pages resident on a different numa nodes", which is exactly what happens in our case.
This leads to very spourius results.
Normally, on HPC systems that parameter is set 0 so to get a predictable behaviour and avoid memory pages traffic among the memory banks.
However, since I had the root permissions on those nodes, I could set it to zero an get the nice matrix presented in Sec. 2.
What if — as normally is — you don't have the root permissions?
Well, you ask for a binding of the memory pages, which come bundled with the usage of `numa_alloc_onnode()` but could be obtained independently by calling the

`mbind()`

function from the `numalib`.

6. Requiring large pages

The default page size on small systems is typically `4KB`; on large facilities, where you are supposed to allocate tens or hundreds of GB, it is more practical to have large pages and less TLB pollution.
You can inquire the page size from inside your code by using
`size_t sysconf(_SC_PAGESIZE);`

as implemented in the function `probe_memory_alloc( )`.
You can also ask the kernel to provide large-pages allocation by using `mmap()` directly instead of an `alloc`-family function, or to merge small pages by using `madvise()` function.

Their usage example is in the final version `test_bandwidth.large_pages.c`.

7. Measuring the latency

The previous design, that at the base of `test_bandwidth.*.c` series, aimed at getting the maximum possible bandwidth: no strided access, subsequent access perfectly predictable by the hardware prefetcher that can completely hide the latency.
Since we wanted to measure the DRAM bandwidth and not the cache bandwidth, in addition we:

- used a memory buffer larger than L3 (well, we hard-coded 256MB, which is reasonable; however, it would be better to parse the output of `lscpu` or even the `/sys/devices/system/cpu/cpu0/cache/index?` files).
- warmed-up the TLB by initializing the arrays (which also was about touching the memory).

However, if our goal is to measure the *latency*, we need to design an experiment that defeats the hardware prefetcher; as you can imagine, a randomized linked-list approach.
What we do is:

- allocate the memory (all the same that we have seen before still holds);
- initialize it (first-touch, useful when not using numa_alloc) to warm-up the TLB, so we do not measure the TLB misses.
- set-up a linked-list of randomized, out-of-page jumps to traverse the allocated memory in a close loop that can be repeated indefinitely.
- measure the time taken by repeated walk, derive the average and the std. deviation and print out the latency matrix.

That is what we give in `test_latency.c`; with this explanation, all the implementation details should be clear enough.

8. What's next

We have seen how to use quite low-level function calls and techniques to deal with memory allocation in OpenMP.
However, the lowest-level you are the more likely is that you need to modify the code, or use `#ifdef` forest to adapt the code to different platforms. What we have seen is quite standard and ubiquitous on Linux systems, though.

OpenMP offers you high-level tools to specify *what kind* of memory you want to use and *how to allocate it*: the **memory spaces** and the **memory allocators**.
Although the compilers' support is still incomplete and fragmented at the moment if writing this note, an overview is given in the document `lecture-notes—openmp_memory_allocators`.

Appendix: the `stream` routine details

Our stream routine is implemented as follows:

```
double stream( const double * restrict array,
               unsigned int N,
               double *stddev,
               double *result )
```

```
{
    unsigned int _N  = N&(~((UNROLLING_FACTOR)-1)); // renders clear that _N is a
                                                    // multiple of UNROLLING_FACTOR
    const double * restrict _a = (const double*) __builtin_assume_aligned( array, 64 );

    #if !defined(UNROLL_FACTOR)
    #define UNROLL_FACTOR 8
    #endif

    double S[UNROLL_FACTOR] = {0};

    double timing = CPU_TIME_T;
   #pragma GCC ivdep
    for ( unsigned int i = 0; i < _N; i += UNROLL_FACTOR )
    {
        for ( int j = 0; j < UNROLL_FACTOR; j++)
        S[j] += _a[i+j]*2.0 + 1.0;
    }
    timing = CPU_TIME_T - timing;
    ...
}
```

There are some implementation details worth noticing for an HPC lecture.

1. note our maximal effort in signaling to the compiler that there is no memory aliasing: the target array `a` is declared `restrict` and `const`
2. to convince the compiler that our data are aligned to 64 bytes, which is needed for AVX512 vector instructions, we use `const double * restrict _a = (const double*) __builtin_assume_aligned( array, 64 );`
3. to convince the compiler that our iteration space is a multiple of the UNROLLING_FACTOR we use `unsigned int _N  = N&(~((UNROLLING_FACTOR)-1));`
4. note that we use `_a` and `_N` in the loop body
5. not to fall in the IEEE math trap we use as many accumulators as UNROLLING_FACTOR, rendering the iterations vectorisable
6. `#pragma GCC ivdep` tells the compiler to ignore any iteration-carried dependency it may spot

Also note that in the complete implementation we are repeating the kernel loop, as reported above, `NREPS` times, taking the average and the std deviation, discarding the two worst results.